第27课 前缀和与差分入门

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：求前缀和（入门）

题目：输入 n 个整数，再输入 q 次查询，每次给出 l 和 r，求区间 [l, r] 内所有数的和。

数据范围：1 ≤ n, q ≤ 100000，数组元素绝对值不超过 10000。

讲解思路：

1. 先读入数组 `a[1..n]`。
2. 构建前缀和数组 `sum[0..n]`，其中 `sum[0] = 0`。
3. 对于每次查询，输出 `sum[r] - sum[l-1]`。

> 引导学生思考：如果不用前缀和，每次查询都要 for 循环累加，q 次查询最坏要算 n × q = 100000 × 100000 = 100 亿次——电脑会超时！用前缀和，构建一次 O(n)，每次查询 O(1)，总共 O(n + q)，快了太多太多！

完整代码：

```
#include <iostream>
using namespace std;

int a[100005];
long long sum[100005];  // 前缀和可能很大，用 long long

int main() {
    int n, q;
    cin >> n >> q;

    // 读入数组（下标从 1 开始）
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    // 构建前缀和数组
    sum[0] = 0;
    for (int i = 1; i <= n; i++) {
        sum[i] = sum[i - 1] + a[i];
    }

    // 处理查询
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << sum[r] - sum[l - 1] << endl;
    }

    return 0;
}
```

逐行讲解要点：

- `long long sum[]`：前缀和累加可能超过 int 范围，养成好习惯用 `long long`。
- `sum[0] = 0`：哨兵！保证 `l = 1` 时公式也正确。
- `sum[i] = sum[i-1] + a[i]`：核心递推，"今天的余额 = 昨天的余额 + 今天存的钱"。
- `sum[r] - sum[l-1]`：区间和公式，一步到位。

例题 2：判断哪段区间和最大（前缀和应用）

题目：小明记录了 n 天的气温变化值（可正可负），他想知道**连续 k 天**的气温变化之和最大是多少。输入 n、k 和 n 个整数，输出最大的连续 k 天之和。

数据范围：1 ≤ k ≤ n ≤ 100000。

讲解思路：

1. 先构建前缀和数组。
2. 连续 k 天的区间就是 [i, i+k-1]，用 `sum[i+k-1] - sum[i-1]` 求和。
3. 枚举所有起点 i（从 1 到 n-k+1），取最大值。

完整代码：

```
#include <iostream>
using namespace std;

int a[100005];
long long sum[100005];

int main() {
    int n, k;
    cin >> n >> k;

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    // 构建前缀和
    sum[0] = 0;
    for (int i = 1; i <= n; i++) {
        sum[i] = sum[i - 1] + a[i];
    }

    // 枚举所有长度为 k 的区间，找最大和
    long long maxSum = sum[k] - sum[0];  // 先假设第一个区间最大
    for (int i = 2; i + k - 1 <= n; i++) {
        long long cur = sum[i + k - 1] - sum[i - 1];
        if (cur > maxSum) {
            maxSum = cur;
        }
    }

    cout << maxSum << endl;

    return 0;
}
```

讲解要点：

- 初始值 `maxSum = sum[k] - sum[0]`，即第一个区间 [1, k] 的和。
- 循环条件 `i + k - 1 <= n`，确保区间不越界。

- 这道题展示了前缀和的经典用法——**把 O(nk) 优化到 O(n)**。

例题 3：差分——区间加分（入门）

题目：班级有 n 个同学，编号 1 到 n，初始成绩都是 0。老师进行了 m 次操作，每次给编号 l 到 r 的同学每人加 v 分。求最终每个同学的成绩。

数据范围：1 ≤ n, m ≤ 100000，1 ≤ v ≤ 100。

讲解思路：

1. 用差分数组 `d[]` 记录变化。
2. 每次操作：`d[l] += v`，`d[r+1] -= v`。
3. 最后对 `d[]` 做一遍前缀和，得到每个同学的最终成绩。

> 回忆"水龙头"比喻：每次操作就是在 l 处开水龙头、r+1 处关水龙头。

完整代码：

```
#include <iostream>
using namespace std;

int d[100005];  // 差分数组

int main() {
    int n, m;
    cin >> n >> m;

    // m 次操作
    while (m--) {
        int l, r, v;
        cin >> l >> r >> v;
        d[l] += v;        // 开水龙头
        d[r + 1] -= v;    // 关水龙头
    }

    // 对差分数组做前缀和，还原最终成绩
    for (int i = 1; i <= n; i++) {
        d[i] = d[i] + d[i - 1];  // 前缀和还原
        cout << d[i];
        if (i < n) cout << " ";
    }
    cout << endl;

    return 0;
}
```

逐行讲解要点：

- 差分数组初始全为 0（因为初始成绩都是 0）。
- `d[l] += v` 和 `d[r+1] -= v`：只操作两个位置，非常快！
- 最后的 for 循环：对差分数组做前缀和 = 还原回原数组。
- 这道题展示了差分的核心用法——**把 m 次区间加的 O(nm) 优化到 O(n + m)**。

手动模拟（板书演示 n=5 的例子）：

```
操作1：l=2, r=4, v=5     d：0 +5  0  0 -5  0
操作2：l=1, r=3, v=3     d：+3 +5  0 -3 -5  0

前缀和还原：
  d[1] = 3
  d[2] = 3+5 = 8
  d[3] = 8+0 = 8
  d[4] = 8+(-3) = 5
  d[5] = 5+(-5) = 0

最终成绩：3  8  8  5  0

验证：
  同学1：被操作2加了3分   3
  同学2：被操作1加5分，操作2加3分   8
  同学3：被操作1加5分，操作2加3分   8
  同学4：被操作1加5分   5
  同学5：没被加过   0
```

附：例题源代码与运行结果

例题 1：Prefix Sum - Range Query

```
// Given n integers and q queries, for each query (l, r),
// output the sum of elements in range [l, r].
#include <iostream>
using namespace std;

int a[100005];
long long sum[100005];  // prefix sum may be large, use long long

int main() {
    int n, q;
    cin >> n >> q;

    // read array (1-indexed)
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    // build prefix sum array
    sum[0] = 0;
    for (int i = 1; i <= n; i++) {
        sum[i] = sum[i - 1] + a[i];
    }

    // answer queries
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << sum[r] - sum[l - 1] << endl;
    }

    return 0;
}
```

Sample Input

```
5 3
400 350 380 420 390
2 4
1 5
3 3
```

Expected Output

```
1150
1940
380
```

例题 2：Maximum Sum of Consecutive K Days

```
// Given n daily temperature changes (positive or negative),
// find the maximum sum of any consecutive k days.
#if 0
#include <iostream>
using namespace std;

int a[100005];
long long sum[100005];

int main() {
    int n, k;
    cin >> n >> k;

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    // build prefix sum
    sum[0] = 0;
    for (int i = 1; i <= n; i++) {
        sum[i] = sum[i - 1] + a[i];
    }

    // enumerate all windows of length k, find the maximum sum
    long long maxSum = sum[k] - sum[0];  // first window [1, k]
    for (int i = 2; i + k - 1 <= n; i++) {
        long long cur = sum[i + k - 1] - sum[i - 1];
        if (cur > maxSum) {
            maxSum = cur;
        }
    }

    cout << maxSum << endl;

    return 0;
}
#endif
```

Sample Input

```
6 3
-2 5 3 -1 4 2
```

Expected Output

```
7
```

例题 3：Difference Array - Range Addition

```
// n students (numbered 1 to n), initial scores are all 0.
// After m operations, each adding v to students in range [l, r],
// output the final score of each student.
#if 0
#include <iostream>
using namespace std;

int d[100005];  // difference array

int main() {
    int n, m;
    cin >> n >> m;

    // m operations
    while (m--) {
        int l, r, v;
        cin >> l >> r >> v;
        d[l] += v;        // open the faucet
        d[r + 1] -= v;    // close the faucet
    }

    // compute prefix sum of difference array to restore final scores
    for (int i = 1; i <= n; i++) {
        d[i] = d[i] + d[i - 1];  // prefix sum to restore
        cout << d[i];
        if (i < n) cout << " ";
    }
    cout << endl;

    return 0;
}
#endif
```

Sample Input

```
5 2
2 4 5
1 3 3
```

Expected Output

```
3 8 8 5 0
```